

Web and Data Engineering

Lecture 04: HTTP und RESTful APIs

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

HTTP ist das Protokoll des Webs. REST nutzt dieses Protokoll konsequent als Grundlage für API-Design. Diese Vorlesung verbindet beides: vom **Protokollverständnis** über **Caching und Sicherheit** bis zum **Entwurf robuster, evolvierbarer APIs**.

Leitfragen

- Warum ist HTTP mehr als Datentransport?
- Wie steuern Header Caching, Sessions und Sicherheit?
- Was unterscheidet REST von beliebigem JSON-über-HTTP?
- Wie modelliert man Ressourcen, URLs und HTTP-Semantik für APIs?
- Wie entwickelt man APIs weiter, ohne Clients zu brechen?

HTTP Grundlagen

Leitfrage: Wie können Browser und Server mit einem einfachen, textbasierten Protokoll so kommunizieren, dass das Web weltweit skalierbar bleibt?

Was sehen Sie hier? {.smaller}

Ein Nutzer ruft die Produktseite auf. Das Netzwerk-Tab zeigt diesen Austausch:

↑ REQUEST

```
GET /api/products/42 HTTP/1.1
Host: shop.example.com
Accept: application/json
Cookie: session=abc123
```

↓ RESPONSE

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=300
ETag: "f7e3a1b2"

{"id":42,"name":"Funkkopfhörer","price":79.99}
```

Aufgabe: Was ist Startzeile, Header, Body? Welche Information steckt in jedem Header?

Teil	Request	Response
Startzeile	GET /api/products/42 HTTP/1.1	HTTP/1.1 200 OK
Header	Host, Accept, Cookie	Content-Type, Cache-Control, ETag
Body	kein Body bei GET	JSON-Daten des Produkts

Cache-Control: max-age=300

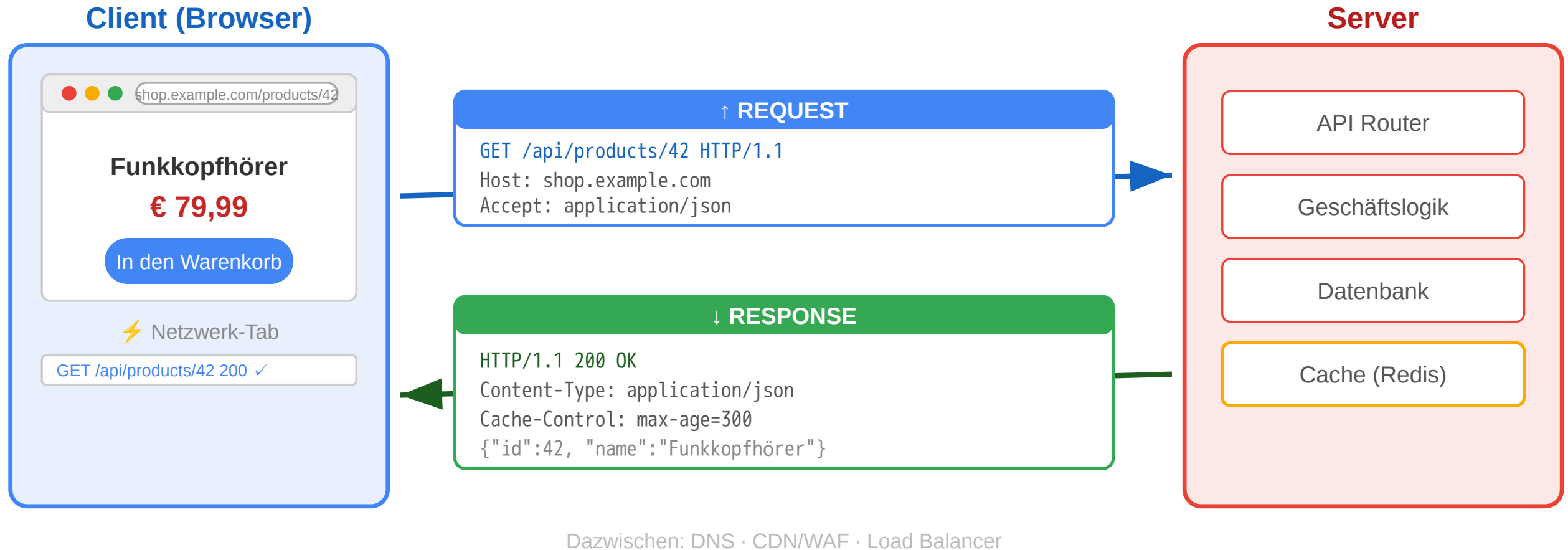
Daten 5 min gültig

ETag: "f7e3a1b2"

Fingerprint zur Validierung



HTTP: Das Request-Response-Protokoll



Von der Fetch API zum Protokoll

In Vorlesung 03 haben wir die **Client-Seite** gesehen:

```
const response = await fetch('/api/products/42', {  
  headers: { 'Accept': 'application/json' }  
});
```

Jetzt schauen wir auf die **Protokollseite**:

Fetch API	HTTP-Protokoll
<code>fetch(url)</code>	Browser erzeugt HTTP-Request mit Startzeile + Headern
<code>{ method: 'POST' }</code>	HTTP-Methode — definiert Absicht und Semantik
<code>{ headers: {...} }</code>	Request-Header — Metadaten für Server und Infrastruktur
<code>response.status</code>	Statuscode — standardisiertes Ergebnis
<code>response.json()</code>	Response Body parsen — Repräsentation der Ressource

Die Fetch API ist eine **JavaScript-Abstraktion über HTTP**. Wer HTTP versteht, versteht auch, warum fetch so funktioniert wie es funktioniert — und kann Netzwerkprobleme im DevTools-Netzwerk-Tab

diagnostizieren.

HTTP-Designprinzipien {.smaller}

Prinzip	Bedeutung	Konsequenz
Ressourcenorientiert	Jede Information hat eine Adresse (URI)	Einheitliche Adressierung weltweit
Request-Response	Client fragt, Server antwortet	Klare Rollenverteilung
Zustandslos	Jeder Request ist unabhängig	Server muss keinen Kontext halten → skaliert
Erweiterbar	Header erlauben beliebige Metadaten	Caching, Auth, Content-Negotiation ohne Protokolländerung
Textbasiert	Nachrichten sind menschenlesbar (HTTP/1.1)	Einfach zu debuggen

Warum das skaliert

- Zustandslosigkeit erlaubt Load Balancing über beliebig viele Server
- Proxies und Caches können HTTP-Nachrichten ohne Anwendungswissen verarbeiten

URI (Uniform Resource Identifier)

`https://shop.example.com:443/api/products/42?lang=de#details`

Schema → Host → Port → Pfad → Query → Fragment

- Klare Trennung von Transport und Anwendungslogik

HTTP-Methoden als semantischer Vertrag {.smaller}

Methode	Absicht	Sicher?	Idempotent?	Online-Shop-Beispiel
GET	Ressource lesen	ja	ja	Produktseite anzeigen
POST	Neue Ressource oder Verarbeitung	nein	nein	Bestellung aufgeben
PUT	Ressource vollständig ersetzen	nein	ja	Profil aktualisieren
PATCH	Ressource teilweise ändern	nein	nein*	Adresse ändern
DELETE	Ressource entfernen	nein	ja	Konto löschen
HEAD	Nur Header, kein Body	ja	ja	Existenzprüfung
OPTIONS	Erlaubte Methoden abfragen	ja	ja	CORS Preflight

Sicher (Safe):

Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.

Idempotent:

Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

HTTP-Methoden: Safe und Idempotent in der Praxis {.smaller}

Szenario: Ein Nutzer klickt zweimal schnell auf „Jetzt kaufen“. Netzwerk-Lag verzögert den ersten Request, beide kommen beim Server an.

Methode	Effekt beim Doppel-Request	Warum?
POST /api/orders	Zwei Bestellungen entstehen	Nicht idempotent — jeder Aufruf erzeugt neue Ressource
PUT /api/cart/item/42	Eine Änderung — zweiter Call ohne Effekt	Idempotent — gleiches Ergebnis egal wie oft

Konsequenz: Load Balancer und API-Gateways können idempotente Requests bei Timeout sicher wiederholen. CDNs cachen nur **safe** Methoden (GET, HEAD), weil diese die Ressource nie verändern.

HTTP-Statuscodes {.smaller}

Klasse	Bedeutung	Wichtige Codes
1xx	Information	101 Switching Protocols
2xx	Erfolg	200 OK, 201 Created, 204 No Content
3xx	Umleitung	301 Moved Permanently, 304 Not Modified
4xx	Client-Fehler	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 429 Too Many Requests
5xx	Server-Fehler	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Was stimmt an dieser API-Antwort nicht?



```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": false,
  "error": "Produkt nicht gefunden"
}
```

Problem: Statuscode lügt

- 200 OK signalisiert Erfolg
- Body enthält aber einen Fehler
- CDNs cachen die „Erfolg“-Antwort
- Monitoring zählt 200 als OK

Richtig:

404 Not Found

als Statuscode, Fehlerdetails im Body.

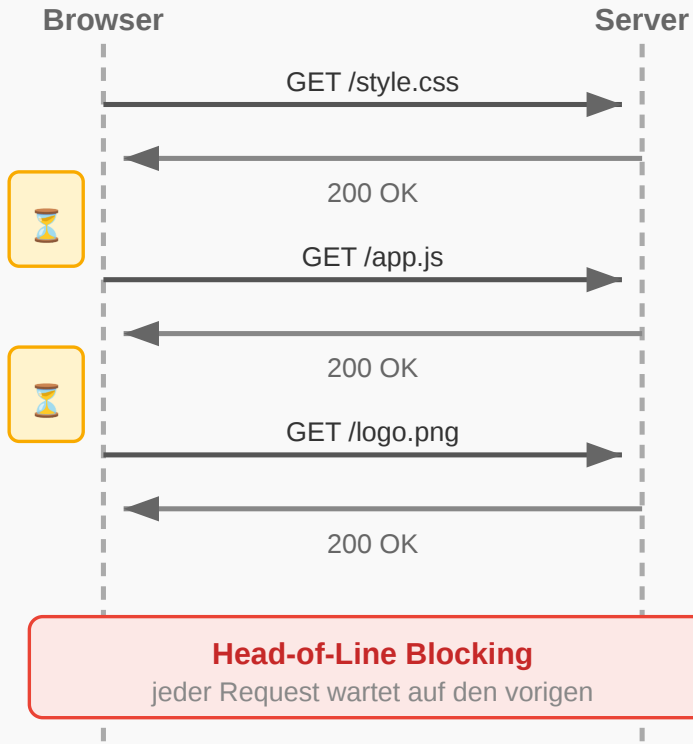
HTTP-Versionen im Vergleich {smaller}

Version	Jahr	Kernverbesserung
HTTP/1.0	1996	Ein Request pro TCP-Verbindung
HTTP/1.1	1997	Persistent Connections, Pipelining
HTTP/2	2015	Multiplexing, Header-Kompression, binär
HTTP/3	2022	QUIC (UDP-basiert), schnellerer Aufbau



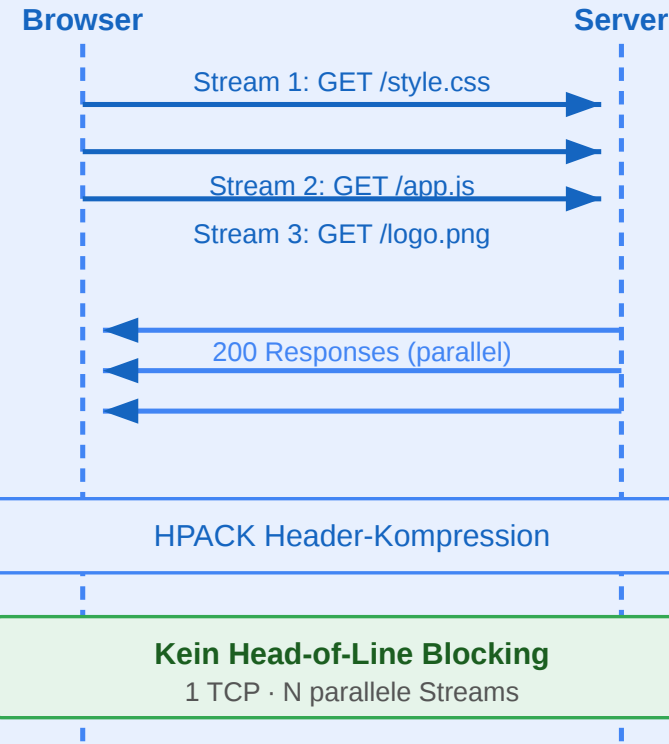
HTTP/1.1

Sequentiell · TCP



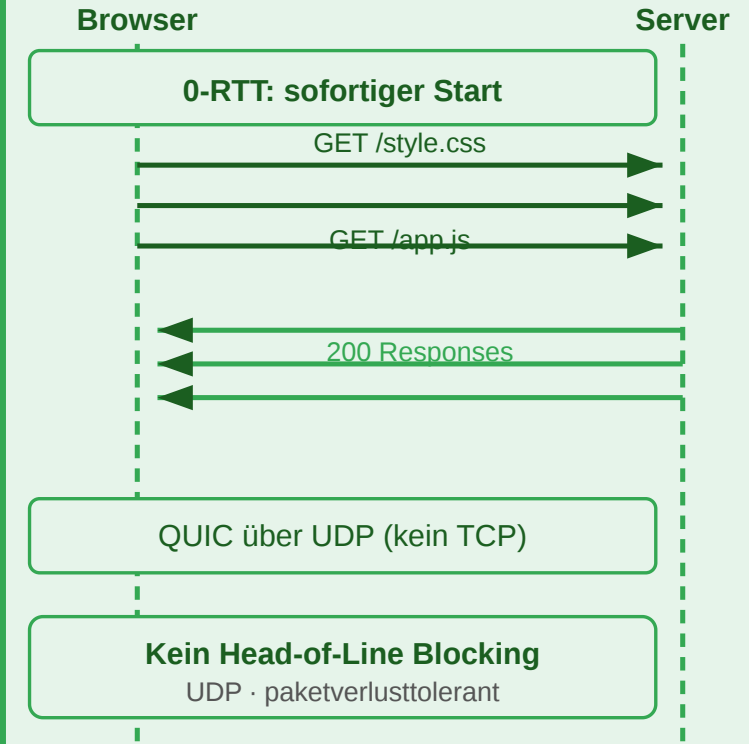
HTTP/2

Multiplexing · TCP



HTTP/3

QUIC · UDP · 0-RTT



Frage: Eine Produktseite lädt 60 Ressourcen (JS, CSS, Bilder). Warum dauert das mit HTTP/1.1 deutlich länger als mit HTTP/2 — obwohl die Bandbreite dieselbe ist?

HTTP Kontrolle, Caching und Sicherheit

Leitfrage: Wie bleibt HTTP trotz Zustandslosigkeit effizient und steuerbar, wenn Inhalte umgeleitet, zwischengespeichert oder erneut validiert werden müssen?

Szenario: Der veraltete Preis {.smaller}

Ein Nutzer besucht die Produktseite des Funkkopfhörers. Der Preis wird von € 79,99 auf € 69,99 **gesenkt**. Der Nutzer lädt die Seite erneut — sieht aber weiterhin den alten Preis.

Aufgabe: Was ist passiert? Welche HTTP-Mechanismen sind beteiligt?

Erklärung

Der Browser hat die vorherige Response gecacht. Der Cache-Control: max-age=300 Header sagte: „Diese Daten sind 5 Minuten gültig.“ Innerhalb dieser Frist fragt der Browser **gar nicht** beim Server nach. Der Nutzer sieht den alten Preis, bis der Cache abläuft.

Architekturentscheidung: Wie lange sollen Produktdaten gecacht werden? Zu lang → veraltete Preise. Zu kurz → jeder Seitenaufruf belastet den Server.

Caching: Ebenen und Mechanismen {.smaller}

Cache-Ebene	Wo?	Vorteil
Browser-Cache	Lokal auf dem Gerät	Kein Netzwerk-Request → schnellste Variante
Proxy/CDN-Cache	Edge-Server im Netzwerk	Reduziert Last auf Origin, niedrige Latenz
Application-Cache	Im Server (Redis, Memcached)	Vermeidet wiederholte Datenbankabfragen

Cache-Steuerung über Header {smaller}

Header	Funktion	Beispiel
Cache-Control: max-age=3600	1 Stunde gültig	Statische Assets
Cache-Control: no-cache	Immer validieren	Produktdaten, Preise
Cache-Control: no-store	Nie cachen	Bankdaten
Cache-Control: public	CDN darf cachen	Öffentliche Inhalte
Cache-Control: private	Nur Browser	Nutzerdaten
ETag	Ressourcen-Fingerprint	"a1b2c3d4"
Last-Modified	Letzter Änderungszeitpunkt	Mon, 23 Mar 2026

Wie der Browser entscheidet:

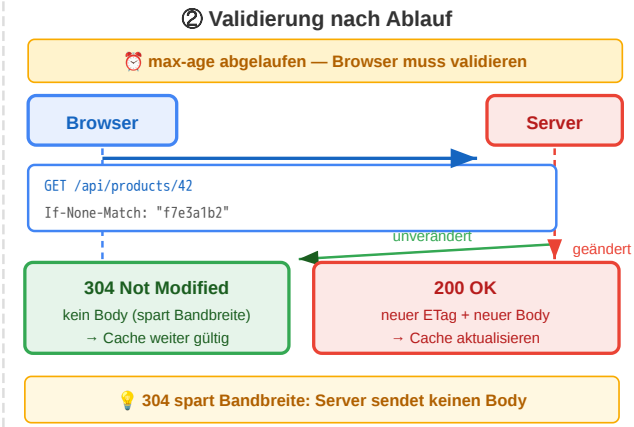
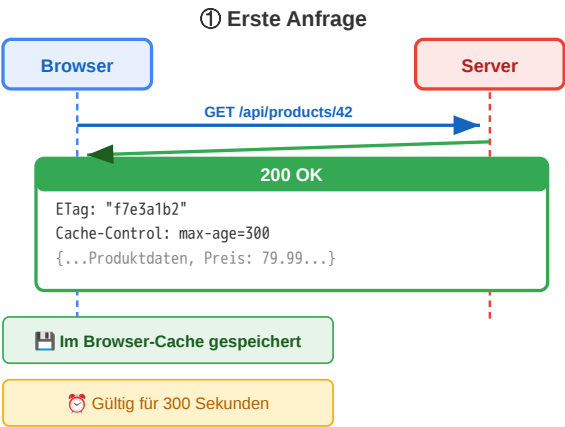
Cache vorhanden? → max-age abgelaufen? → Validierungsrequest mit

If-None-Match: ETag

Validierung:

Server antwortet

304 Not Modified



(kein Body, spart Bandbreite) oder
200 OK
+ neuer Body + neuer ETag.

Interpretationsaufgabe: Caching-Strategie {.smaller}

Aufgabe: Welchen Cache-Control-Header würden Sie für diese Ressourcen des Online-Shops setzen?

Ressource	Ihr Vorschlag?
app.v3.2.1.js (versionierte JS-Datei)	
/api/products/42 (Produktdaten mit Preis)	
/api/cart (Warenkorb des eingeloggten Nutzers)	
logo.png (Shop-Logo)	

Ressource	Header	Begründung
app.v3.2.1.js	public, max-age=31536000, immutable	Version im Dateinamen → ändert sich nie
/api/products/42	public, no-cache + ETag	Preise können sich ändern, Validierung nötig
/api/cart	private, no-store	Nutzerspezifisch, darf nicht im CDN landen
logo.png	public, max-age=86400	Ändert sich selten, 1 Tag reicht

Redirects und das PRG-Pattern {.smaller}

```
Browser — POST /api/orders —> Server
Server — 303 See Other —> Browser
Browser — GET /orders/789 —> Server
Server — 200 OK (Bestellbestätigung) —> Browser
```

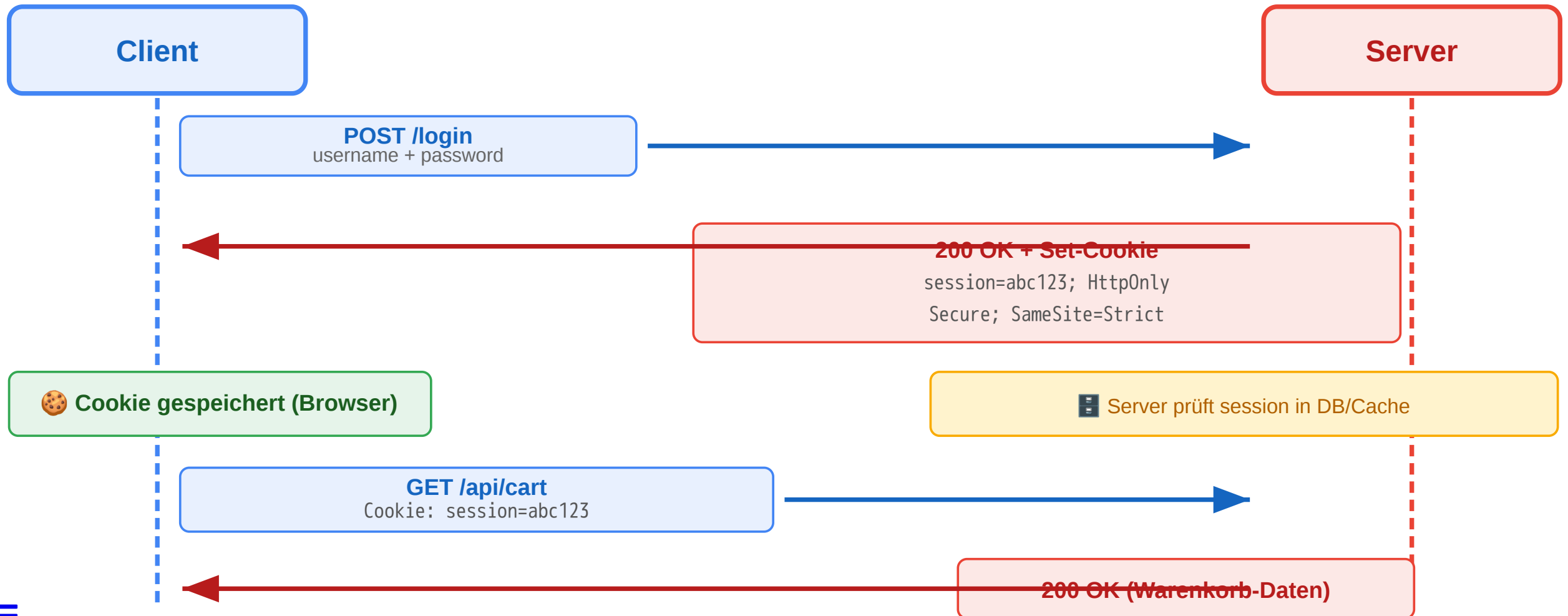
Code	Semantik	Typischer Einsatz
301	Permanent verschoben	Domain-Umzug
302/307	Temporär woanders	A/B-Test, Wartung
303	Nach POST auf GET	PRG-Pattern
308	Permanent, Methode bleibt	API-Migration

PRG verhindert doppelte Bestellungen:

Nach POST antwortet Server mit 303 → Browser folgt per GET → Neuladen der Seite wiederholt nur den GET, nicht den POST.

Sessions und Cookies {.smaller}

HTTP ist zustandslos — aber der Online-Shop braucht Login und Warenkorb. **Cookies** lösen diesen Widerspruch:



Welche Cookie-Konfiguration ist sicher?

Konfiguration	XSS-sicher?	CSRF-sicher?	HTTPS-only?
session=abc123	nein	nein	nein
session=abc123; HttpOnly	ja	nein	nein
session=abc123; HttpOnly; Secure	ja	nein	ja
session=abc123; HttpOnly; Secure; SameSite=Strict	ja	ja	ja

Nur die letzte Variante schützt gegen die drei häufigsten Angriffsvektoren.

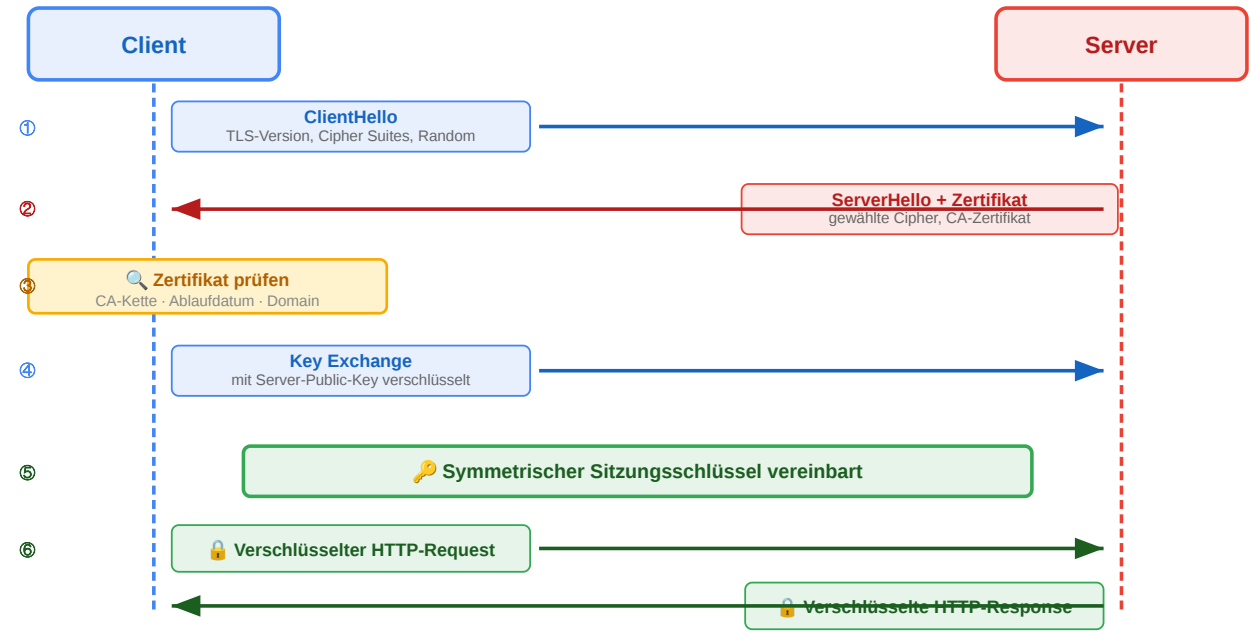
HTTPS und TLS {.smaller}

Eigenschaft	HTTP	HTTPS
Vertraulichkeit	Klartext lesbar	Verschlüsselt
Integrität	Manipulierbar	Manipulation erkannt
Authentizität	Keine Garantie	Zertifikat bestätigt

Diskussionsfrage:

Ein Angreifer im selben WLAN liest den Netzwerkverkehr mit. Welche der drei TLS-Eigenschaften schützt hier — und welche nicht?

(Hinweis: TLS schützt den Transport, nicht den Endpunkt.)



HTTP bleibt simpel auf Protokollebene, wird aber durch Header und Statuscodes zum ausdrucksstarken Steuerungsmechanismus. Caching ist der wichtigste Performance-Hebel — aber jede Caching-Entscheidung ist ein Trade-off zwischen Aktualität und Last. Redirects steuern

Navigation, PRG verhindert Doppelverarbeitung. Cookies ermöglichen Zustand trotz Zustandslosigkeit
— aber nur mit korrekter Konfiguration. TLS sichert den Transport.

REST als Architekturidee

Leitfrage: Was unterscheidet eine RESTful API von einem beliebigen HTTP-Endpunkt, der nur JSON ausliefert?

Eine typische AI-Antwort auf diese Frage {smaller}

„REST ist ein Architekturstil für APIs, bei dem man JSON über HTTP verschickt. Man benutzt GET zum Lesen, POST zum Erstellen, PUT zum Aktualisieren und DELETE zum Löschen. Die URLs sollten Ressourcen beschreiben, z.B. /users/42.“

Aufgabe: Was ist an dieser Antwort korrekt? Was fehlt? Was ist irreführend?

Analyse

Aussage	Bewertung
„JSON über HTTP“	Irreführend — REST ist formatunabhängig. JSON ist verbreitet, aber nicht Voraussetzung.
„GET zum Lesen, POST zum Erstellen...“	Teilweise korrekt — beschreibt CRUD-Mapping, aber nicht die eigentlichen REST-Constraints



Aussage	Bewertung
„URLs sollten Ressourcen beschreiben“	Korrekt — aber nur ein Aspekt der Uniform Interface
Fehlt: Statelessness	Kritisch — Zustandslosigkeit ist der Grund, warum REST skaliert
Fehlt: Cacheability	Kritisch — ein Hauptvorteil gegenüber RPC-Ansätzen
Fehlt: Layered System	Wichtig — ermöglicht CDN, Load Balancer, API Gateway transparent einfügbar

Die AI-Antwort beschreibt die **Syntax** von REST, aber nicht die **Architektur**. Das ist der häufigste Fehler.

REST: Ein Architekturstil, kein Protokoll {.smaller}

Constraint	Bedeutung	Konsequenz
Client-Server	Klare Rollentrennung	Client und Server entwickeln sich unabhängig
Stateless	Jeder Request enthält alle nötigen Informationen	Server skaliert horizontal, kein Session-Zustand
Cacheable	Responses deklarieren ihre Cachebarkeit	Infrastruktur kann Caching transparent umsetzen
Uniform Interface	Einheitliche Schnittstelle für alle Ressourcen	Generische Werkzeuge (Browser, curl, Proxies) funktionieren
Layered System	Client weiß nicht, ob er direkt mit dem Server spricht	Load Balancer, CDN, API Gateway transparent einfügbar
Code on Demand (optional)	Server kann ausführbaren Code liefern	JavaScript im Browser

Kernidee



REST nutzt die **vorhandene Web-Infrastruktur** (HTTP, URIs, Caching, Proxies) als Anwendungsplattform — statt ein eigenes RPC-Protokoll darüber zu bauen.

Welchen Constraint verletzt dieses Design? {.smaller}

Szenario 1: Der Online-Shop speichert den Warenkorb in einer serverseitigen Session. Beim nächsten Request muss der Client denselben Server treffen (Sticky Sessions).

Szenario 2: Die API liefert bei GET /api/products keinen Cache-Control-Header. Jeder Request geht bis zum Origin-Server.

Szenario 3: Der Client kennt die interne Datenbankstruktur und baut SQL-artige Queries: GET /api/query?table=products&where=price>50

Szenario	Verletzter Constraint	Konsequenz
1: Sticky Sessions	Stateless	Horizontale Skalierung eingeschränkt, Server-Ausfall = Session verloren
2: Kein Cache-Control	Cacheable	CDN nutzlos, unnötige Serverlast, höhere Latenz
3: SQL-artige Queries	Uniform Interface + Layered System	Client an Interna gekoppelt, keine Abstraktion möglich

REST vs. RPC im Vergleich {.smaller}

RPC-Stil (aktionsorientiert)

```
POST /createUser
POST /getUser
POST /updateUser
POST /deleteUser
POST /getUserOrders
POST /sendPasswordReset
```

- Jeder Endpunkt ist eine **Funktion**
- HTTP-Methode immer POST
- Infrastruktur kann nichts cachen
- Client muss Doku lesen

REST-Stil (ressourcenorientiert)

```
POST   /users           → anlegen
GET    /users/42        → lesen
PUT    /users/42        → ersetzen
DELETE /users/42        → löschen
GET    /users/42/orders → Bestellungen
POST   /password-resets → anstoßen
```

- Jeder Endpunkt ist eine **Ressource**
- HTTP-Methoden drücken Absicht aus
- GET-Requests sind cachebar
- Verhalten ist vorhersagbar

RPC-Stil

Aktions-orientiert

POST /createUser

alles POST

POST /getUser

POST /updateUser

POST /deleteUser

- ✗ CDN kann nichts cachen
- ✗ Monitoring sieht nur POST
- ✗ WAF kann nicht filtern
- ✗ Client muss Doku lesen
- ✗ Jeder Endpunkt ist eine Funktion

vs.

REST-Stil

Ressourcen-orientiert

POST /users → anlegen

GET /users/42 → lesen

PUT /users/42 → ersetzen

DELETE /users/42 → löschen

- ✓ GET ist cachebar (CDN)
- ✓ Methoden in Audit-Logs sichtbar
- ✓ WAF erkennt DELETE, POST
- ✓ Verhalten ist vorhersagbar
- ✓ Jeder Endpunkt ist eine Ressource

Warum das für Infrastruktur relevant ist



Ressourcen und Repräsentationen {.smaller}

Eine **Ressource** ist ein fachliches Konzept. Was der Client empfängt, ist eine **Repräsentation**.

Konzept	Bedeutung
Ressource	Das fachliche Ding (existiert im System)
URI	Die Adresse der Ressource (identifiziert sie eindeutig)
Repräsentation	Die konkrete Darstellung (JSON, HTML, CSV — je nach Accept-Header)
Zustandsübergang	Aktion auf der Ressource (GET liest, POST erzeugt, DELETE entfernt)

Content Negotiation: Client und Server einigen sich über Accept und Content-Type auf das Format — dieselbe Ressource, verschiedene Repräsentationen.

Wann stößt REST an Grenzen? {.smaller}

Situation	REST-Problem	Alternative
Client braucht Daten aus 5 Ressourcen gleichzeitig	5 separate Requests → Latenz	GraphQL: Ein Query, ein Response
Echtzeit-Updates (Chat, Live-Ticker)	Polling ist ineffizient	WebSockets: bidirektionale Verbindung
Interne Microservice-Kommunikation	JSON-Overhead, keine Typsicherheit	gRPC: binäres Protokoll, Protobuf-Schema
Komplexe Aktionen (Batch-Operationen)	Passt nicht in CRUD	RPC-Endpunkte als Ergänzung

Kein Entweder-Oder

Viele Systeme **kombinieren** Stile: REST für öffentliche APIs, GraphQL für flexible Frontends, gRPC für interne Services. Die Wahl hängt von **Nutzungsszenarien** ab — nicht von Dogma.



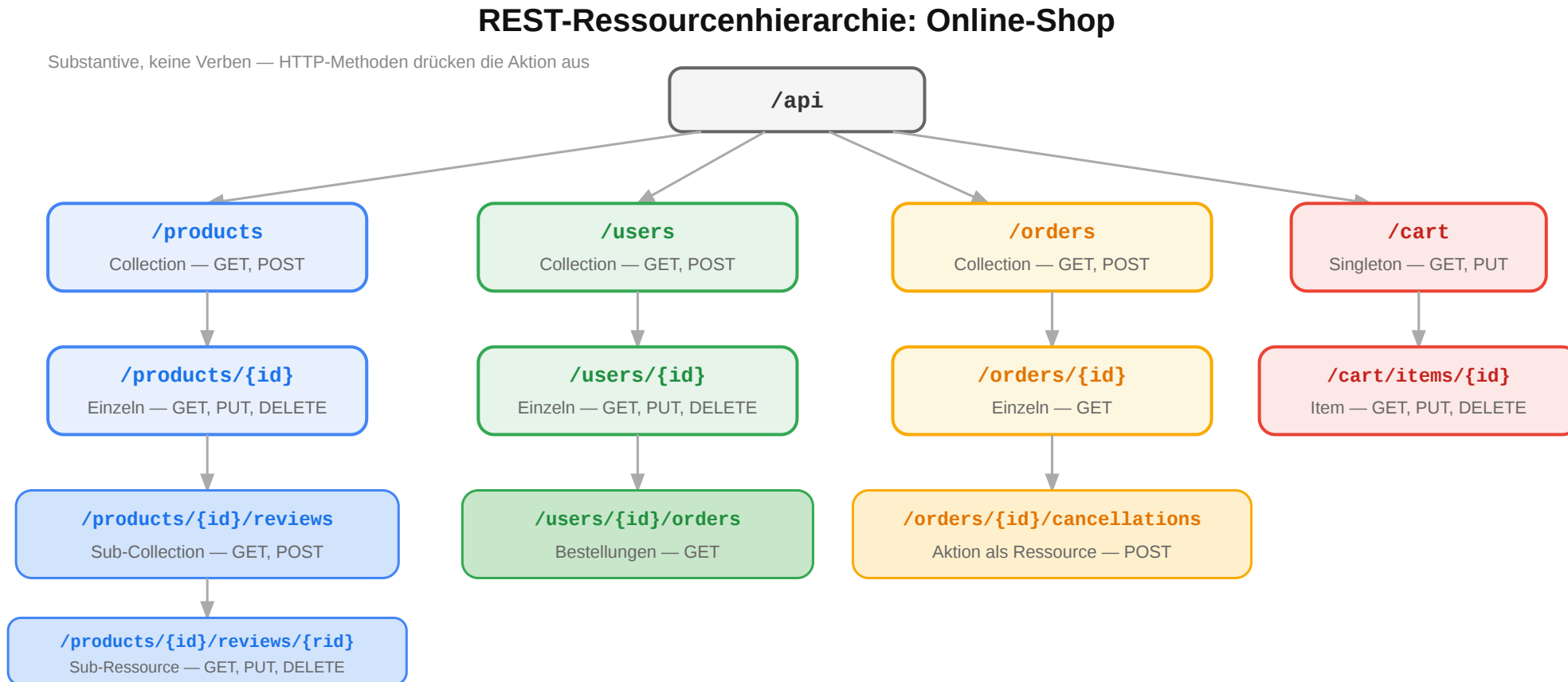
REST ist kein JSON-over-HTTP. Es ist ein Architekturstil mit sechs Constraints, die das Web skalierbar, cachebar und erweiterbar machen. Die Stärke liegt in der konsequenten Nutzung bestehender Web-Infrastruktur. Wer die Constraints versteht, kann beurteilen, wann REST die richtige Wahl ist und wann ein anderer Stil besser passt.

Ressourcen und URI-Design

Leitfrage: Wie werden fachliche Objekte so als Ressourcen modelliert, dass eine API verständlich und stabil bleibt?

Ressourcenorientiertes Design: Online-Shop {.smaller}

Der erste Schritt beim API-Design ist die Identifikation der **Ressourcen** — die fachlichen Dinge, mit denen Clients interagieren:



URI-Design-Regeln {.smaller}

Regel	Gut	Schlecht	Warum
Plural für Collections	/products	/product	Collection enthält viele
Kleinschreibung, Bindestriche	/order-items	/OrderItems	URL-Konvention
Keine Verben im Pfad	POST /orders	POST /createOrder	Methode drückt Aktion aus
Hierarchie = Zugehörigkeit	/users/42/orders	/getUserOrders? userId=42	Beziehung sichtbar
Query für Filter/Sortierung	/products? sort=price	/products/sortByPrice	Pfad = Identität, Query = Parameter

Anti-Patterns erkennen

```
POST /api/getUser      → ?
POST /api/deleteAccount → ?
GET  /api/search?action=find → ?
POST /api/doPayment    → ?
```



Übung: API-Design für eine Domäne {.smaller}

Aufgabe: Entwerfen Sie die REST-Ressourcen für ein **Universitäts-Kursverwaltungssystem**. Welche Ressourcen gibt es? Welche URIs? Welche Hierarchien?

Hinweis: Denken Sie an Kurse, Vorlesungen, Studierende, Einschreibungen, Noten.

Eine mögliche Lösung

Fachliches Konzept	URI	Methoden
Alle Kurse	/courses	GET, POST
Ein Kurs	/courses/web-eng	GET, PUT, DELETE
Vorlesungen eines Kurses	/courses/web-eng/lectures	GET, POST
Einschreibungen	/courses/web-eng/enrollments	GET, POST
Eine Einschreibung	/courses/web-eng/enrollments/42	GET, DELETE
Noten eines Studierenden	/students/42/grades	GET



Fachliches Konzept	URI	Methoden
Note in einem Kurs	/students/42/grades/web-eng	GET, PUT

Diskussion: Sollte die Note unter /students/42/grades/web-eng oder unter /courses/web-eng/grades/42 liegen? Beide sind valide — die Wahl hängt davon ab, **wer der primäre Nutzer der API ist.**

CRUD-Mapping auf HTTP {.smaller}

Operation	HTTP	URI	Body	Response
Liste lesen	GET	/products	—	200 + Array
Einzeln lesen	GET	/products/42	—	200 + Objekt
Anlegen	POST	/products	Neues Objekt	201 + Location
Vollständig ersetzen	PUT	/products/42	Vollständig	200 / 204
Teilweise ändern	PATCH	/products/42	Nur Änderungen	200 / 204
Löschen	DELETE	/products/42	—	204

Jenseits von CRUD

Nicht alles passt in CRUD. Für **Aktionen** gibt es zwei Muster:

- **Sub-Ressource:** POST /orders/42/cancellations — Stornierung als neue Ressource
- **Controller-Ressource:** POST /password-resets — Aktion als eigene Ressource

Pagination, Filter und Sortierung {.smaller}

```
GET /products?page=2&per_page=20&sort=-price&category=electronics&q=kopfhörer
```

Parameter	Funktion	Beispiel
page / per_page	Seitenbasierte Pagination	?page=3&per_page=25
cursor	Cursor-basierte Pagination	?cursor=eyJpZCI6NDJ9
sort	Sortierung (- = absteigend)	?sort=-created_at,name
filter	Filterung	?status=active&min_price=10
q	Volltextsuche	?q=kopfhörer

Pagination in der Response

```
{
  "data": [{"id": 42, "name": "Funkkopfhörer", "price": 79.99}],
  "meta": { "total": 142, "page": 2, "per_page": 20 },
  "links": {
    "next": "/products?page=3&per_page=20",
    "prev": "/products?page=1&per_page=20"
  }
}
```



Links machen die API navigierbar — der Client muss keine URIs konstruieren.

Gute REST-APIs beginnen mit sauber geschnittenen Ressourcen. URI-Design ist eine **Modellierungsaufgabe**: Ressourcen identifizieren, Beziehungen als Hierarchie abbilden, Aktionen über HTTP-Methoden ausdrücken. Die Entscheidung, wo eine Ressource in der Hierarchie steht, hängt vom Nutzungskontext ab — nicht von der Datenbankstruktur.

HTTP-Semantik für APIs

Leitfrage: Wie helfen Methoden, Statuscodes und Header dabei, API-Verhalten ohne Zusatzprotokoll klar und maschinenlesbar zu machen?

Was ist an dieser API-Antwort falsch? {.smaller}

Ein Client sendet eine Bestellung an den Online-Shop:

```
POST /api/orders HTTP/1.1
Content-Type: application/json

{"product_id": 42, "quantity": 2}
```

Der Server antwortet:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id": 789, "status": "created"}
```

Aufgabe: Die Bestellung wurde erfolgreich erstellt. Trotzdem hat die Response mindestens drei Probleme. Welche?

Problem	Warum	Besser
200 OK statt 201 Created	200 sagt „gelesen“, nicht „erzeugt“	201 Created
Kein Location-Header	Client weiß nicht, wo die neue Ressource liegt	Location: /api/orders/789



Problem	Warum	Besser
Kein Cache-Control	Bestellungen dürfen nicht gecacht werden	Cache-Control: no-store

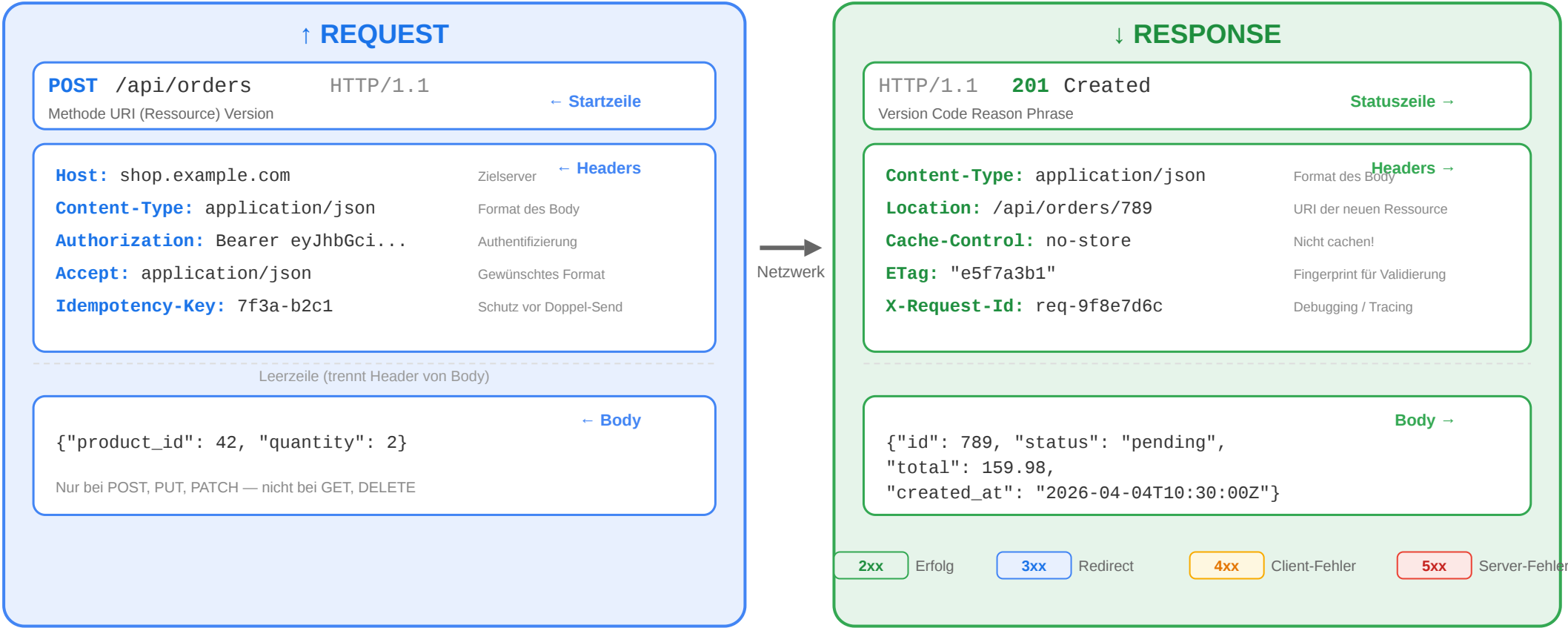
Korrigierte Version

```
HTTP/1.1 201 Created
Location: /api/orders/789
Content-Type: application/json
Cache-Control: no-store

{"id": 789, "status": "pending", "total": 159.98}
```

Der vollständige Request-Response-Zyklus {smaller}

Anatomie: HTTP Request und Response



Statuscodes richtig einsetzen {.smaller}

Situation	Richtiger Code	Häufiger Fehler
Ressource erfolgreich gelesen	200 OK	—
Ressource erfolgreich angelegt	201 Created + Location	200 ohne Location
Erfolgreich, aber kein Body	204 No Content	200 mit leerem Body
Ungültige Eingabedaten	400 Bad Request	500 (ist kein Server-Fehler!)
Nicht authentifiziert	401 Unauthorized	403
Authentifiziert, aber nicht berechtigt	403 Forbidden	401
Ressource nicht gefunden	404 Not Found	200 mit Fehlermeldung im Body
Konflikt (z.B. doppelter Username)	409 Conflict	400

Faustregel

Der Statuscode sagt dem Client, **was passiert ist**. Der Body erklärt **warum** und **was zu tun ist**.

Fehlermodelle: Konsistenz ist alles {.smaller}

```
{
  "error": {
    "type": "validation_error",
    "message": "Die Eingabedaten sind ungültig.",
    "details": [
      { "field": "email", "code": "invalid_format",
        "message": "Keine gültige E-Mail-Adresse" },
      { "field": "quantity", "code": "out_of_range",
        "message": "Muss zwischen 1 und 100 liegen" }
    ]
  }
}
```

Prinzip	Umsetzung
Konsistente Struktur	Immer dasselbe JSON-Format für Fehler
Maschinenlesbar	type und code für automatische Verarbeitung
Menschenlesbar	message für Entwickler und Endnutzer
Spezifisch	details pro Feld bei Validierungsfehlern
Kein Stacktrace	Interne Details nie an den Client

Authentifizierung: Wer darf was? {smaller}

Mechanismus	Funktionsweise	Typischer Einsatz
API Key	Statischer Schlüssel im Header	Einfache M2M-Kommunikation
Bearer Token (JWT)	Signierter Token im Authorization-Header	SPAs, Mobile Apps
OAuth 2.0	Delegierte Autorisierung über Authorization Server	„Login mit Google“
Session Cookie	Cookie mit Session-ID	Klassische server-gerenderte Apps

JWT (JSON Web Token)

```
Header.Payload.Signature  
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjo0Mn0.abc123signature
```

Teil	Inhalt
Header	Algorithmus und Token-Typ
Payload	Claims: user_id, Rollen, Ablaufzeit
Signature	Kryptografische Signatur → Manipulation erkennbar



Diskussionsfrage: JWT ist stateless — der Server muss keine Session speichern. Aber: Wie widerruft man einen JWT, bevor er abläuft?

Rate Limiting und Idempotenz {.smaller}

Idempotenz: Warum doppelt senden nicht doppelt bestellen sollte

Methode	Idempotent?	Warum
GET	ja	Liest nur
PUT	ja	Ergebnis ist immer derselbe Zustand
DELETE	ja	Zweites Löschen hat keinen Effekt
POST	nein	Jeder Aufruf kann neue Ressource erzeugen

Szenario: Ein Nutzer klickt „Jetzt bestellen“, die Verbindung bricht ab. Hat der Server die Bestellung erhalten? Der Client weiß es nicht und sendet erneut.

Lösung: Idempotency-Key-Header — der Server erkennt den Retry anhand des Schlüssels und liefert das Ergebnis des ersten Requests zurück, statt eine zweite Bestellung anzulegen.

Rate Limiting



```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 0
```

APIs begrenzen Requests pro Zeitfenster. Die Header sagen dem Client **wann** er es erneut versuchen darf.

HTTP-Semantik ist der Vertrag zwischen Client und Server. Statuscodes kommunizieren Ergebnisse, Header steuern Caching und Authentifizierung, konsistente Fehlermodelle machen APIs nutzbar. Idempotenz-Mechanismen schützen gegen doppelte Verarbeitung. Je klarer die HTTP-Semantik genutzt wird, desto weniger Sonderlogik brauchen Clients.

Evolution, Fehler und API-Qualität

Leitfrage: Wie entwickelt man APIs weiter, ohne bestehende Clients bei jeder Änderung zu brechen – und wie misst man die Qualität einer API?

Welche dieser Änderungen brechen Clients? {smaller}

Der Online-Shop plant ein API-Update. Bewerten Sie jede Änderung:

Geplante Änderung	Kompatibel?
Neues Feld "created_at" in Produkt-Response	
Feld "discount" aus Produkt-Response entfernen	
Neuer optionaler Query-Parameter ?include=reviews	
Feld "userName" umbenennen in "user_name"	
Neuer Endpunkt GET /products/{id}/recommendations	
Pflichtfeld "phone" in Registrierung hinzufügen	

Änderung	Kompatibel?	Begründung
Neues Feld in Response	ja	Clients ignorieren unbekannte Felder
Feld aus Response entfernen	nein	Client erwartet "discount", Feld fehlt → Fehler



Änderung	Kompatibel?	Begründung
Neuer optionaler Parameter	ja	Bestehende Requests funktionieren weiter
Feld umbenennen	nein	Client liest "userName", findet es nicht
Neuer Endpunkt	ja	Bestehende Endpunkte unverändert
Pflichtfeld hinzufügen	nein	Bestehende Clients senden "phone" nicht → 400

Prinzip: Additive Evolution

Neue Felder, Endpunkte, optionale Parameter → kompatibel. Entfernen, Umbenennen, Required machen → Breaking Change.

API-Lebenszyklus {.smaller}

Phase	Herausforderung
Design	Ressourcen richtig schneiden, ohne zu viel vorwegzunehmen
Implement	Server + Client SDK konsistent halten
Test	Nicht nur „funktioniert es?“, sondern „hält der Vertrag?“
Release	Dokumentation und Versionierung synchron halten
Operate	Fehler erkennen, bevor Clients sie melden
Evolve	Neue Anforderungen einbauen, ohne Bestehendes zu brechen

Robustheitsprinzip (Postel's Law)

„Be liberal in what you accept, conservative in what you send.“



Diskussionsfrage: Wenn der Server „liberal“ ungültige Eingaben akzeptiert, entsteht ein impliziter Vertrag. Ist das wirklich eine gute Idee?

Deprecation-Strategie {.smaller}

Wenn Breaking Changes unvermeidbar sind, müssen sie **angekündigt** und **schrittweise** eingeführt werden:

```
Deprecation: true  
Sunset: Sat, 01 Nov 2026 00:00:00 GMT  
Link: </docs/migration-v3>; rel="deprecation"
```

Header	Funktion
Deprecation: true	Endpunkt als veraltet markiert
Sunset	Ab diesem Datum wird entfernt
Link	Verweis auf Migrationsdokumentation

API-Dokumentation mit OpenAPI {.smaller}

```
openapi: 3.0.3
info:
  title: Shop API
  version: 1.0.0
paths:
  /products/{id}:
    get:
      summary: Produkt abrufen
      parameters:
        - name: id
          in: path
          required: true
          schema:
            type: integer
      responses:
        '200':
          description: Produkt gefunden
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Product'
```

Vorteil

Erklärung

Maschinenlesbar

Automatische Code-Generierung

Interaktive Docs

Swagger UI — Entwickler können direkt testen



Vorteil	Erklärung
Contract Testing	Schema als Grundlage für automatisierte Tests
Single Source of Truth	Dokumentation und Implementierung synchron

Observability: APIs im Betrieb {.smaller}

Die vier goldenen Signale

Signal	Frage	Online-Shop-Beispiel
Latency	Wie lange dauern Requests?	p95 für GET /products < 200ms
Traffic	Wie viel Last hat das System?	5.000 Requests/s zur Cyber-Monday-Spitze
Errors	Wie hoch ist die Fehlerrate?	0,1% 5xx ist akzeptabel
Saturation	Wie nah an der Kapazitätsgrenze?	DB-Connection-Pool bei 85%

Säule	Was wird gemessen?	Werkzeuge
Logging	Einzelne Requests: Wer, Was, Wann, Ergebnis	ELK Stack, Loki
Metriken	Aggregiert: Requests/s, Latenz, Fehlerrate	Prometheus, Grafana
Tracing	Ein Request über alle Services hinweg	Jaeger, OpenTelemetry

Bewerten Sie diese API {smaller}

Aufgabe: Prüfen Sie die API des Online-Shops gegen diese Checkliste. Was fehlt?

```
GET /products      → 200, kein Cache-Control
POST /products     → 200 statt 201, kein Location
GET /products/999  → 200 mit {"error": "not found"}
POST /orders       → 500 bei ungültiger Eingabe
DELETE /products/42 → 200 mit Body
Fehler-Format: mal {"error": "..."}, mal {"message": "..."}

```

Problem	Verstoß	Korrektur
Kein Cache-Control	Caching unkontrolliert	Cache-Control: no-cache + ETag
200 statt 201 bei POST	Falscher Statuscode	201 Created + Location
200 bei nicht gefunden	Statuscode lügt	404 Not Found
500 bei ungültiger Eingabe	Server-Fehler ≠ Client-Fehler	400 oder 422
Body bei DELETE	Unnötig	204 No Content
Inkonsistentes Fehlerformat	Client kann Fehler nicht parsen	Einheitliches Error-Schema



API-Design endet nicht beim ersten Release. Die eigentliche Qualität zeigt sich in **Evolution** (additive Changes, Deprecation-Strategie, Postel's Law), **Fehlerkultur** (konsistente Fehlermodelle, spezifische Statuscodes) und **Betriebsfähigkeit** (Observability mit den vier goldenen Signalen). Eine gute API ist vorhersagbar, dokumentiert, beobachtbar und entwickelt sich weiter, ohne bestehende Clients zu brechen.

Wrap-Up

- **HTTP** ist das Interaktionsmodell des Webs: Methoden drücken Absichten aus, Statuscodes kommunizieren Ergebnisse, Header steuern Caching, Sessions und Sicherheit.
- **REST** nutzt diese HTTP-Mechanismen konsequent für APIs — Ressourcen statt Funktionsaufrufe, einheitliche Schnittstelle statt proprietärer Protokolle.
- **Gutes API-Design** beginnt bei sauber geschnittenen Ressourcen, nutzt HTTP-Semantik vollständig und plant Evolution von Anfang an mit.
- **Betrieb** erfordert Observability, Rate Limiting und konsistente Fehlermodelle.

Die nächste Vorlesung zeigt, wie diese APIs **serverseitig implementiert** werden — am Beispiel des Java-Web-Stacks.